

с4.0 Какой путь сейчас доказываем

Время: 00:00–04:00

ЭКРАН: доска, полный путь запроса; затем браузер DEVOPS MAY CRY.

ГОВОРЮ

«В первом видео путь от браузера до Pod'a мы не разбирали. Сейчас пройдем его по слоям и на каждом найдем объект, который можно проверить.

Сначала разберем TCP, DNS и маршруты. Внутри кластера посмотрим Pod resolv.conf и CoreDNS. Затем свяжем NodePort, Service, EndpointSlice и текущий data plane kube-proxy. В финале добавим внешний этаж: два HAProxy, keepalived, VRRP и один VIP. Остановим активный балансировщик и измерим, что увидел клиент.

Конкретный лабораторный путь такой: браузер → локальный SSH-туннель лабы → private VIP `10.77.0.10:8080` → HAProxy L4 → NodePort `30080` → Service/EndpointSlice → Pod. В production туннеля нет: его место занимает реальный public IP и маршрутизация. Ingress в эту цепочку не вставлен: сначала доказываем базовый L4-маршрут без второго контроллера».

ПЕРЕХОД

«До Kubernetes начну с соединения на одном компьютере. Там три пакета handshake видны без облака и CNI».

C4.1 TCP: SYN, SYN/ACK, ACK

Время: 04:00–11:00

ЭКРАН: два окна терминала или один блок с tcpdump в фоне.

ГОВОРЮ

«TCP сначала создаёт соединение. Клиент отправляет SYN, сервер отвечает SYN/ACK, клиент подтверждает ACK. Только после этого идёт HTTP.

Запускаю обычный Python HTTP server на loopback и ограниченный tcpdump. Это не наше production-приложение — здесь нужен только прозрачный TCP-поток».

V2-C4-S01-C01 · Увидеть TCP handshake на loopback

```
REPO_ROOT="$(git rev-parse --show-toplevel)"
cd "$REPO_ROOT"
sudo -v
sudo tcpdump -i lo0 -n 'tcp port 8080' &
TCPDUMP_PID=$!
python3 -m http.server 8080 >/tmp/video2-http.log 2>&1 &
HTTP_PID=$!
sleep 1
curl -fsS http://127.0.0.1:8080/ >/dev/null
sleep 1
sudo kill -INT "$TCPDUMP_PID" 2>/dev/null || true
wait "$TCPDUMP_PID" || true
netstat -an | grep -E 'LISTEN|ESTABLISHED|TIME_WAIT' | head
kill "$HTTP_PID" 2>/dev/null || true
```

После команды

Ожидая: в tcpdump видны SYN → SYN/ACK → ACK, затем HTTP-пакеты и закрытие соединения; `netstat` показывает состояния TCP.

ЧТО ЭТО ЗНАЧИТ

`LISTEN` относится к серверному socket, `ESTABLISHED` — к активному соединению, `TIME_WAIT` защищает от старых сегментов после закрытия. Kubernetes эти состояния не отменяет: NodePort и Service всё равно перевозят TCP.

ПЕРЕХОД

«TCP доставляет байты. Без TLS эти байты можно прочитать».

с4.2 Base64 в HTTP не защищает пароль

Время: 11:00–17:00

ЭКРАН: ASCII tcpdump → Authorization header → base64 decode.

ГОВОРЮ

«Basic Authentication кладёт `user:pass` в base64. Это удобная кодировка для текста, не шифрование. Если HTTP идёт без TLS, заголовков виден в capture».

V2-C4-S02-C01 · Доказать, что Basic base64 читается из трафика

```
python3 -m http.server 8080 >/tmp/video2-http.log 2>&1 &
HTTP_PID=$!
sleep 1
sudo -v
sudo tcpdump -i lo0 -n -A 'tcp port 8080' &
TCPDUMP_PID=$!
sleep 1
curl -s http://127.0.0.1:8080/secret -H 'Authorization: Basic dXNlcjpwYXNz'
>/dev/null
sleep 1
sudo kill -INT "$TCPDUMP_PID" 2>/dev/null || true
wait "$TCPDUMP_PID" || true
printf 'dXNlcjpwYXNz' | base64 -d; echo
kill "$HTTP_PID" 2>/dev/null || true
```

После команды

Ожидая: ASCII capture содержит HTTP-заголовок Authorization; base64 декодируется в `user:pass`. Это кодирование, не шифрование.

В ПРОДЕ

TLS защищает канал, но не делает слабый пароль сильным и не отменяет срок жизни credentials. Внутри кластера шифрование трафика зависит от CNI/service mesh и политики, а не появляется само по себе.

ПЕРЕХОД

«Чтобы подключиться по имени, клиент сначала должен получить адрес. Разделю системный resolver и прямой DNS-запрос».

с4.3 hosts, системный resolver и dig — не одно и то же

Время: 17:00–25:00

ЭКРАН: hosts → scutil DNS → dig → временная строка myapp.local.

ГОВОРЮ

«Приложение обычно вызывает системный resolver. Он может учитывать `/etc/hosts`, кэш и настройки ОС. `dig` отправляет DNS-запрос напрямую и `/etc/hosts` не читает.

На минуту добавляю `myapp.local` в hosts. `ping` его находит, `dig` — нет. После доказательства строку сразу удаляю».

V2-C4-S03-C01 · Сравнить системный resolver и прямой DNS-запрос

```
cat /etc/hosts
scutil --dns | sed -n '1,90p'
dig example.com A
dig +trace example.com A
echo '127.0.0.1 myapp.local' | sudo tee -a /etc/hosts
ping -c1 myapp.local
dig myapp.local +short
sudo sed -i '' '/myapp.local/d' /etc/hosts
```

После команды

Ожидая: системный resolver находит `myapp.local` через hosts, а `dig` обращается прямо к DNS и ничего о hosts не знает; строка после доказательства удалена.

ЛОВУШКА

Не оставлять тестовую строку в hosts. И не доказывать работу DNS с помощью `ping`: он показывает итог системного resolver, а не обязательно DNS-ответ.

ПЕРЕХОД

«Теперь поймаю сам DNS-запрос. Resolver и интерфейс возьму из текущей системы, а не захардкожу публичный 8.8.8.8».

с4.4 DNS capture без отключения VPN и перестройки routes

Время: 25:00–31:00

ЭКРАН: фактический resolver, route interface, query/answer на port 53.

ГОВОРЮ

«Сначала читаю, какой nameserver реально использует macOS, затем спрашиваю route, через какой интерфейс до него идти. На машине с VPN это может быть `utun`. Ничего не выключаю и не перенастраиваю — только наблюдаю».

V2-C4-S04-C01 · Снять DNS-пакеты через фактический resolver без изменения VPN

```
DNS_SERVER="$(scutil --dns | awk '/nameserver\[([0-9]+\)/{{print $3; exit}}')"
```

```
test -n "$DNS_SERVER"
```

```
IFACE="$(route -n get "$DNS_SERVER" | awk '/interface:/{print $2; exit}')
```

```
printf 'resolver=%s interface=%s\n' "$DNS_SERVER" "$IFACE"
```

```
sudo -v
```

```
sudo tcpdump -i "$IFACE" -n "host $DNS_SERVER and port 53" &
```

```
TCPDUMP_PID=$!
```

```
sleep 1
```

```
dig @"$DNS_SERVER" example.com A
```

```
sleep 1
```

```
sudo kill -INT "$TCPDUMP_PID" 2>/dev/null || true
```

```
wait "$TCPDUMP_PID" || true
```

После команды

Ожидая: запрос и ответ DNS видны на интерфейсе реального route. VPN, DNS и системные routes не меняются.

ЧТО ЭТО ЗНАЧИТ

DNS отвечает за имя → адрес. Таблица маршрутов отвечает, через какой interface/gateway отправить пакет к этому адресу. Это разные этапы.

ПЕРЕХОД

«Адрес получили. Теперь покажу выбор маршрута на ноутбуке и между двумя VM».

C4.5 Longest prefix match: какой route победил

Время: 31:00–39:00

ЭКРАН: адреса стенда → `route get` → SSH на node2 → private route до node3.

ГОВОРЮ

«Public SSH-адреса рабочего стенда уже записаны в локальном recording env. Private IP не угадываю и не копирую из UI — снимаю командой на самой VM.

`route get` показывает выбранный interface и gateway. Linux-команда `ip route get` делает то же для конкретного destination. Правило выбора — самый длинный подходящий префикс».

V2-C4-S05-C01 · Проследить route с ноутбука и private route между VM

```
set -a
source "$REPO_ROOT/recording/.recording.env"
set +a
SSH_KEY="${VIDEO2_SSH_KEY:?fill VIDEO2_SSH_KEY in recording/.recording.env}"
SSH_USER="${VIDEO2_SSH_USER:-root}"
NODE1_PUBLIC_IP="${NODE1_SSH_HOST:?fill NODE1_SSH_HOST}"
NODE2_PUBLIC_IP="${NODE2_SSH_HOST:?fill NODE2_SSH_HOST}"
NODE3_PRIVATE_IP="$(ssh -i "$SSH_KEY" "$SSH_USER@${NODE3_SSH_HOST:?fill
NODE3_SSH_HOST}" \
    "ip -4 route get 1.1.1.1 | awk '{for (i=1;i<=NF;i++) if (\$i==\"src\") {print
\$(i+1); exit}}'"")"
route -n get default
route -n get "$NODE1_PUBLIC_IP"
traceroute -m 5 "$NODE1_PUBLIC_IP" || true
ssh -i "$SSH_KEY" "$SSH_USER@$NODE2_PUBLIC_IP" \
    "ip -4 route; ip route get '$NODE3_PRIVATE_IP'"
```

✅ После команды

Ожидаю: ноутбук выбирает interface/gateway по таблице маршрутов; node2 достигает private IP node3 по VPC route. Никакой маршрут вручную не добавляется.

🚧 В ПРОДЕ

Если приложение недоступно только из одной среды, сравнивают route, DNS, security group/ACL и source IP. Ручное добавление маршрута на ноутбуке — не первый диагностический шаг и не замена исправлению инфраструктурного кода.

ПЕРЕХОД

«Переношу тот же вопрос внутрь Pod: какой DNS ему выдали и почему короткое имя Service работает».

с4.6 Pod resolv.conf, CoreDNS, search и ndots

Время: 39:00–49:00

ЭКРАН: debug-client YAML → CoreDNS Pod'ы → `/etc/resolv.conf` клиента.

ГОВОРЮ

«Применяю DEVOPS MAY CRY и отдельный debug-client с `dig` и `curl`. Debug-инструменты не добавляю в distroless production image: для диагностики создаю отдельный одноразовый Pod».

 **V2-C4-S06-C01 · Запустить приложение и debug-client, затем открыть Pod resolv.conf**

```
set -a
source "$REPO_ROOT/recording/.recording.env"
set +a
CTX="${SELF_CONTEXT:-kubespray}"
LAB="$REPO_ROOT/04_ЧАСТЬ_4_TRAFFIC/38_4.6_pod_resolv_coredns"
kubectl --context "$CTX" apply -f "$LAB/devops_may_cry.yaml"
kubectl --context "$CTX" -n traffic-lab rollout status deploy/devops-may-cry --
timeout=180s
kubectl --context "$CTX" apply -f "$LAB/debug-client.yaml"
kubectl --context "$CTX" -n traffic-lab wait --for=condition=Ready pod/client --
timeout=180s
kubectl --context "$CTX" -n kube-system get pods -l k8s-app=kube-dns -o wide
kubectl --context "$CTX" -n traffic-lab exec client -- cat /etc/resolv.conf
```

После команды

Ожидая: CoreDNS Pod'ы Running; в client видны cluster DNS Service IP, search suffixes и `options ndots:5`.

ПОКАЗАТЬ

- `nameserver` — ClusterIP kube-dns.
- search suffix текущего namespace, `svc` и cluster domain.
- `ndots:5`: имя с меньшим числом точек сначала пробуется через search list.

ГОВОРЮ

«Полное имя Service — `devops-may-cry.traffic-lab.svc.cluster.local`. Короткое `devops-may-cry` работает внутри того же namespace благодаря search suffix».

V2-C4-S06-C02 · Разрешить полное и короткое имя Service через CoreDNS

```
kubectl --context "$CTX" -n traffic-lab exec client -- \
  dig +short devops-may-cry.traffic-lab.svc.cluster.local
kubectl --context "$CTX" -n traffic-lab exec client -- \
  dig +search +short devops-may-cry
```

После команды

Ожидая: оба запроса возвращают один ClusterIP; короткое имя раскрывается через search текущего namespace.

ПЕРЕХОД

«DNS вернул ClusterIP. Но внешний клиент до ClusterIP не маршрутизируется. Для простого входа открою NodePort».

с4.7 NodePort: порт на нодах, а не новый Pod

Время: 49:00–58:00

ЭКРАН: `kubernetes/devops-may-cry/services/nodeport.yaml` → Service → curl с lb1 на private IP node2:30080.

ГОВОРЮ

«NodePort резервирует один порт на Kubernetes-нодах и связывает его с Service. В нашем каноне порт закреплён как `30080`, потому что позже HAProxy будет использовать его как backend.

Public/floating IP здесь нужен только для SSH. С lb1 проверяю NodePort по private IP node2. Ровно так позже пойдёт HAProxy».

V2-C4-S07-C01 · Открыть NodePort и проверить вход через Kubernetes-ноду

```
kubectl --context "$CTX" apply -f "$REPO_ROOT/kubernetes/devops-may-cry/services/nodeport.yaml"
kubectl --context "$CTX" -n traffic-lab get service devops-may-cry-ha-nodeport -o wide
NODE_PORT="$(kubectl --context "$CTX" -n traffic-lab get svc devops-may-cry-ha-nodeport -o jsonpath='{.spec.ports[0].nodePort}')"
NODE2_PRIVATE_IP="$(ssh -i "$SSH_KEY" "$SSH_USER@${NODE2_SSH_HOST:?fill NODE2_SSH_HOST}" \
    "ip -4 route get 1.1.1.1 | awk '{for (i=1;i<=NF;i++) if (\$i=="src") {print \$(i+1); exit}}'"")"
ssh -i "$SSH_KEY" "$SSH_USER@${LB1_SSH_HOST:?fill LB1_SSH_HOST}" \
    "curl -fsS http://$NODE2_PRIVATE_IP:$NODE_PORT/readyz"
```

✅ После команды

Ожидая: Service показывает NodePort `30080`; запрос с lb1 на private IP node2 получает readiness JSON. Public/floating IP нужен только SSH-клиенту.

⚠ ЛОВУШКА

Доступность public NodePort зависит от cloud security group. `type: NodePort` сам по себе не открывает firewall провайдера.

ПЕРЕХОД

«NodePort ведёт к Service. Теперь разделю стабильный VIP Service и изменяемый список backend'ов».

с4.8 Service остаётся, Pod IP меняются

Время: 58:00–70:00

ЭКРАН: Service → Pod IP → EndpointSlice → удаление одного Pod.

ГОВОРЮ

«Service хранит ClusterIP, ports и selector. Фактические ready backend-адреса публикуются в EndpointSlice. Поэтому ClusterIP — не IP приложения и не адрес одного Pod».

V2-C4-S08-C01 · Связать стабильный ClusterIP с изменяемыми EndpointSlice

```
kubectl --context "$CTX" -n traffic-lab get svc devops-may-cry -o wide
kubectl --context "$CTX" -n traffic-lab get pod -l app=devops-may-cry -o wide
kubectl --context "$CTX" -n traffic-lab get endpointslice \
  -l kubernetes.io/service-name=devops-may-cry -o wide
kubectl --context "$CTX" -n traffic-lab exec client -- curl -fsS http://devops-
may-cry/readyz
```

После команды

Ожидая: ClusterIP отличается от Pod IP; EndpointSlice содержит ready Pod IP; запрос через имя Service получает HTTP 200.

ГОВОРЮ

«Удаляю один Pod. Deployment создаёт замену с новым IP, EndpointSlice обновляется, а ClusterIP не меняется. Именно эта стабильность позволяет клиентам не знать жизненный цикл реплик».

📑 V2-C4-S08-C02 · Удалить Pod и увидеть замену endpoint без смены ClusterIP

```
SERVICE_IP="$(kubectl --context "$CTX" -n traffic-lab get svc devops-may-cry -o
jsonpath='{.spec.clusterIP}')"
POD="$(kubectl --context "$CTX" -n traffic-lab get pod -l app=devops-may-cry -o
jsonpath='{.items[0].metadata.name}')"
OLD_POD_IP="$(kubectl --context "$CTX" -n traffic-lab get pod "$POD" -o
jsonpath='{.status.podIP}')"
DESIRED="$(kubectl --context "$CTX" -n traffic-lab get deploy devops-may-cry -o
jsonpath='{.spec.replicas}')"
kubectl --context "$CTX" -n traffic-lab delete pod "$POD" --wait=true

ENDPOINT_IPS=""
for _ in $(seq 1 180); do
    READY="$(kubectl --context "$CTX" -n traffic-lab get deploy devops-may-cry \
        -o jsonpath='{.status.readyReplicas}')"
    ENDPOINT_IPS="$(kubectl --context "$CTX" -n traffic-lab get endpointslice \
        -l kubernetes.io/service-name=devops-may-cry \
        -o jsonpath='{range .items[*].endpoints[*]}.{.addresses[0]}{"\n"}{end}')"
    ENDPOINT_COUNT="$(printf '%s\n' "$ENDPOINT_IPS" | awk 'NF {n++} END {print
n+0}')"
    if [ "${READY:-0}" = "$DESIRED" ] \
        && [ "$ENDPOINT_COUNT" -ge "$DESIRED" ] \
        && ! printf '%s\n' "$ENDPOINT_IPS" | grep -Fxq "$OLD_POD_IP"; then
        break
    fi
    sleep 1
done
test "${READY:-0}" = "$DESIRED"
test "$ENDPOINT_COUNT" -ge "$DESIRED"
! printf '%s\n' "$ENDPOINT_IPS" | grep -Fxq "$OLD_POD_IP"
kubectl --context "$CTX" -n traffic-lab get endpointslice \
    -l kubernetes.io/service-name=devops-may-cry -o wide
test "$SERVICE_IP" = "$(kubectl --context "$CTX" -n traffic-lab get svc devops-
```

```
may-cry -o jsonpath='{.spec.clusterIP}')"
kubectl --context "$CTX" -n traffic-lab exec client -- curl -fsS http://devops-
may-cry/readyz
```

✓ После команды

Ожидая: bounded-цикл ждёт полного числа ready endpoints, старый Pod IP исчезает, а HTTP снова даёт 200; ClusterIP остаётся тем же.

ПЕРЕХОД

«Осталось понять, где ClusterIP превращается в выбор endpoint. Не буду заранее говорить iptables или IPVS — сначала читаю текущую конфигурацию kube-proxy».

с4.9 kube-проху: сначала факт, потом объяснение

Время: 70:00–80:00

ЭКРАН: kube-proxy ConfigMap → worker IPVS rules → HTTP proof.

ГОВОРЮ

«У нашего Kubespray-стенда выбран IPVS и включён strictARP. Проверяю это в ConfigMap, затем смотрю правила на workers.

В IPVS-режиме ClusterIP появляется на `kube-ipvs0`, а virtual server содержит real servers из EndpointSlice. В iptables или nftables реализация была бы другой; общий контракт Service и EndpointSlice остаётся».

V2-C4-S09-C01 · Доказать текущий kube-proxy data plane через IPVS

```
SERVICE_IP="$(kubectl --context "$CTX" -n traffic-lab get svc devops-may-cry -o
jsonpath='{.spec.clusterIP}')"
kubectl --context "$CTX" -n kube-system get configmap kube-proxy \
  -o jsonpath='{.data.config\.conf}' | grep -E '^mode:|^strictARP:'
ANSIBLE="$REPO_ROOT/kubespray/.venv/bin/ansible"
INV="$REPO_ROOT/kubespray/inventory/video2/inventory.ini"
"$ANSIBLE" -i "$INV" kube_node --private-key "$SSH_KEY" --become \
  -m ansible.builtin.shell \
  -a "ip -brief address show kube-ipvs0; ipvsadm -Ln | grep -F -A4
'${SERVICE_IP}:80'"
kubectl --context "$CTX" -n traffic-lab exec client -- \
  curl -fsS -o /dev/null -w 'service HTTP %{http_code}\n' http://devops-may-
cry/readyz
```

✅ После команды

Ожидая: config сообщает IPVS; на workers VIP находится на `kube-ipvs0`, virtual server указывает на ready endpoints, HTTP возвращает 200.

🧠 ЧТО ЭТО ЗНАЧИТ

Цепочка внутри кластера теперь доказана четырьмя независимыми фактами: Service VIP, ready endpoints, IPVS virtual server и HTTP 200 от приложения.

ПЕРЕХОД

«Раз эта цепочка понятна, сломаю selector одной опечаткой и найду точку разрыва без перезапуска приложения».

Время: 80:00–91:00

ЭКРАН: broken Service с `nigth-shift` → DNS → пустой EndpointSlice → labels.

ГОВОРЮ

«Создаю Service с опечаткой в selector. DNS-запись и ClusterIP появятся, потому что объект Service валиден. Но ни один Pod label ему не соответствует, поэтому ready endpoints нет».

V2-C4-S10-C01 · Получить Service с DNS и ClusterIP, но без endpoints

```
cat <<'EOF' | kubectl --context "$CTX" apply -f -
apiVersion: v1
kind: Service
metadata:
  name: devops-may-cry-broken
  namespace: traffic-lab
spec:
  selector:
    app: nigth-shift
  ports:
    - port: 80
      targetPort: 8080
EOF
kubectl --context "$CTX" -n traffic-lab exec client -- dig +search +short devops-
may-cry-broken
kubectl --context "$CTX" -n traffic-lab get endpointslice \
  -l kubernetes.io/service-name=devops-may-cry-broken -o wide
kubectl --context "$CTX" -n traffic-lab exec client -- \
  curl -m3 -s -o /dev/null -w '%{http_code}\n' devops-may-cry-broken || true
```

✅ После команды

Ожидаю: DNS и ClusterIP существуют, EndpointSlice не содержит ready адресов, HTTP не проходит. Go-приложение при этом остаётся Ready.

🗣️ ГОВОРЮ

«Иду по цепочке с ближайшего проверяемого слоя: DNS → Service selector → Pod labels → EndpointSlice. Ошибка видна до application logs. Исправляю selector, ограниченно жду ready endpoint и повторяю тот же HTTP proof. Контроллеры сходятся не мгновенно, поэтому один ранний `get` не считаю результатом».

📖 V2-C4-S10-C02 · Найти опечатку selector, восстановить endpoints и HTTP

```
kubectl --context "$CTX" -n traffic-lab describe svc devops-may-cry-broken | sed
-n '/Selector:/p'
kubectl --context "$CTX" -n traffic-lab get pods --show-labels
kubectl --context "$CTX" -n traffic-lab patch svc devops-may-cry-broken \
  -p '{"spec":{"selector":{"app":"devops-may-cry"}}}'
READY_ENDPOINTS=0
for _attempt in $(seq 1 30); do
  READY_ENDPOINTS="$(kubectl --context "$CTX" -n traffic-lab get endpointslice \
    -l kubernetes.io/service-name=devops-may-cry-broken \
    -o jsonpath='{range .items[*].endpoints[*]}.{.addresses[0]}{" "}'
    {.conditions.ready}{"\n"}{end}' \
    | grep -c ' true$' || true)"
  test "$READY_ENDPOINTS" -gt 0 && break
  sleep 2
done
test "$READY_ENDPOINTS" -gt 0
kubectl --context "$CTX" -n traffic-lab get endpointslice \
  -l kubernetes.io/service-name=devops-may-cry-broken -o wide
HTTP_CODE=000
for _attempt in $(seq 1 30); do
  HTTP_CODE="$(kubectl --context "$CTX" -n traffic-lab exec client -- \
    curl -sS -o /dev/null -w '%{http_code}' devops-may-cry-broken/readyz ||
true)"
  printf 'ready_endpoints=%s http=%s\n' "$READY_ENDPOINTS" "$HTTP_CODE"
  test "$HTTP_CODE" = "200" && break
  sleep 2
done
test "$HTTP_CODE" = "200"
kubectl --context "$CTX" -n traffic-lab delete svc devops-may-cry-broken
```

✅ После команды

Ожидаю: selector `nighth-shift` не совпадает с label; после исправления endpoints появляются и HTTP становится 200; сломанный Service удалён.

🚧 В ПРОДЕ

Успешный `kubectl apply` доказывает только принятие объекта API. Recovery доказывают Ready workload, ready endpoints и запрос клиента. Если контейнер не стартовал из-за image pull или admission dependency, пустые application logs не опровергают инфраструктурную причину.

ПЕРЕХОД

«Для HTTP-хостов, TLS и маршрутов поверх Service обычно добавляют Ingress Controller или Gateway API. В нашей live-схеме сейчас другой слой: внешний L4-балансировщик напрямую ведёт на NodePort».

Время: 91:00–96:00

ЭКРАН: доска, Ingress/Gateway API между внешним LB и Service; команд нет.

ГОВОРЮ

«Ingress — старый удобный API для HTTP/HTTPS-маршрутов. Работает не сам объект, а установленный Ingress Controller. Gateway API разделяет инфраструктурную точку входа, listener и маршруты приложений; это удобнее для команд с разным владением.

Сегодня я не подменяю этой схемой текущую лабораторию. Здесь HAProxy работает на L4 и ведёт сразу в NodePort. Отдельный Ingress/Gateway слой можно вставить между ними, когда нужны host/path routing и TLS termination».

ПЕРЕХОД

«MetalLB уже показал IP для Service внутри Kubernetes. Теперь уберу его и верну в цепочку внешний HA-этаж на lb1/lb2, который мы установили и проверили в части 1. Здесь он нужен повторно как вход полного browser-to-Pod доказательства».

Время: 96:00–118:00 плюс Ansible

ЭКРАН: NodePort → `inventory.example.ini` и graph без hostvars → site.yml → три роли → PLAY RECAP.

ГОВОРЮ

«MetalLB был отдельной практикой L2 внутри Kubernetes. Перед внешней схемой удаляю его Service и chart, чтобы на доске и в кластере не осталось двух разных точек входа с одним названием.

DEVOPS MAY CRY остаётся. Для HA создаю отдельный NodePort `30080` и проверяю ready endpoints».

📄 V2-C4-S11-C01 · Убрать MetalLB и подготовить NodePort для внешней HA-пары

```
set -a
source "$REPO_ROOT/recording/.recording.env"
set +a
CTX="${SELF_CONTEXT:-kubespray}"
HA_DIR="$REPO_ROOT/ansible/external-ha"
INV="$HA_DIR/inventory.ini"
SSH_KEY="${VIDEO2_SSH_KEY:?fill VIDEO2_SSH_KEY in recording/.recording.env}"
source "$REPO_ROOT/kubespray/.venv/bin/activate"
kubectl --context "$CTX" -n traffic-lab delete service devops-may-cry-metallb --
ignore-not-found
helm --kube-context "$CTX" uninstall metallb -n metallb-system || true
kubectl --context "$CTX" apply -k "$REPO_ROOT/kubernetes/devops-may-cry/base"
kubectl --context "$CTX" -n traffic-lab rollout status deploy/devops-may-cry --
timeout=180s
kubectl --context "$CTX" apply -f "$REPO_ROOT/kubernetes/devops-may-
cry/services/nodeport.yaml"
kubectl --context "$CTX" -n traffic-lab get service devops-may-cry-ha-nodeport -o
wide
kubectl --context "$CTX" -n traffic-lab get endpointslice \
    -l kubernetes.io/service-name=devops-may-cry-ha-nodeport -o wide
```

✅ После команды

Ожидаю: MetalLB Service/chart удалены, приложение Ready, канонический backend — NodePort 30080 с ready endpoints.

🗣️ ГОВОРЮ

«Безопасный inventory example и graph показывают пять ролей без реальных hostvars. В рабочем inventory `ansible_host` — public/floating адрес только для SSH, но этот файл в кадре не печатаю. Ansible facts сам снимает private адрес интерфейса; именно

private IP используется для VXLAN между lb1/lb2 и для backend NodePort на node1-node3».

📄 V2-C4-S11-C02 · Проверить inventory, private IP и backend до применения HA

```
sed -n '1,180p' "$HA_DIR/inventory.example.ini"
ansible-inventory -i "$INV" --graph
ansible -i "$INV" all --private-key "$SSH_KEY" -m ansible.builtin.ping
ansible -i "$INV" all --private-key "$SSH_KEY" -m ansible.builtin.setup \
  -a 'filter=ansible_default_ipv4'
ansible-playbook -i "$INV" "$HA_DIR/preflight-backends.yml" --private-key
"$SSH_KEY"
```

✅ После команды

Ожидая: безопасный example и graph показывают lb1/lb2 и node1-node3 без hostvars; реальные floating-адреса не печатаются. Hosts доступны, facts показывают private IP для VXLAN/backends; preflight с LB достигает private NodePort всех Kubernetes-нод.

🖨️ ПОКАЗАТЬ

- `preflight-backends.yml`: отдельная проверка private NodePort с lb1 до применения ролей.
- `site.yml`: facts всех пяти VM → VXLAN → HAProxy → keepalived → итоговый proof.
- `roles/vxlan_l2`: отдельная лабораторная L2-сеть пары LB.
- `haproxy.cfg.j2`: `mode tcp`, три private backend `:30080`, HTTP health check `/readyz`.
- `keepalived.conf.j2`: `vrrp_script`, `interval/fall`, `nopreempt`, VIP `10.77.0.10/24` на `vxlan100`.

🗣️ ГОВОРЮ

«Сначала syntax check. Затем Ansible напрямую повторно применяет те же роли. На сохранённом стенде это проверка идемпотентности: большинство задач должны дать `ok` или `changed=0`. Если стенд был явно сброшен, те же роли восстановят конфигурацию. Ничего не прячется в install-скрипт».

V2-C4-S11-C03 · Прочитать роли и идемпотентно применить HAProxy + keepalived

```
sed -n '1,240p' "$HA_DIR/site.yml"
sed -n '1,220p' "$HA_DIR/roles/vxlan_l2/tasks/main.yml"
sed -n '1,220p' "$HA_DIR/roles/haproxy/templates/haproxy.cfg.j2"
sed -n '1,220p' "$HA_DIR/roles/keepalived/templates/keepalived.conf.j2"
ansible-playbook -i "$INV" "$HA_DIR/site.yml" --syntax-check --private-key
"$SSH_KEY"
ansible-playbook -i "$INV" "$HA_DIR/site.yml" --private-key "$SSH_KEY"
```

После команды

Ожидая: `PLAY RECAP` без failed. На непрерывном стенде после части 1 большинство задач дают `ok`/`changed=0`; после явного reset те же роли установят конфигурацию заново. HAProxy работает в `mode tcp`, health check делает HTTP `GET /readyz`, keepalived управляет VIP 10.77.0.10 на vxlan100.

ГОВОРЮ

«После зелёного PLAY RECAP проверяю не только systemd. VIP должен быть ровно на одном LB, а HTTP через VIP — 200. Чтобы открыть этот private VIP в браузере, делаю локальный SSH-туннель через lb1. Это только мост учебной сети, а не замена public ingress».

📄 V2-C4-S11-C04 · Доказать сервисы, единственного владельца VIP и HTTP 200

```
ansible -i "$INV" load_balancers --private-key "$SSH_KEY" --become \  
  -m ansible.builtin.command -a 'systemctl is-active haproxy keepalived'  
ansible -i "$INV" load_balancers --private-key "$SSH_KEY" --become \  
  -m ansible.builtin.command -a 'ip -4 addr show dev vxlan100'  
VIP_OWNERS=()  
for host in lb1 lb2; do  
  if ansible -i "$INV" "$host" --private-key "$SSH_KEY" --become \  
    -m ansible.builtin.command -a 'ip -4 addr show dev vxlan100' \  
    | grep -q "$HA_VIP/24"; then  
    VIP_OWNERS+=("$host")  
  fi  
done  
test "${#VIP_OWNERS[@]}" -eq 1  
printf 'VIP owner: %s\n' "${VIP_OWNERS[0]}"  
ansible -i "$INV" lb1 --private-key "$SSH_KEY" --become \  
  -m ansible.builtin.uri -a "url=http://$HA_VIP:$HA_PORT/readyz status_code=200"  
ssh -i "$SSH_KEY" -o ExitOnForwardFailure=yes -N \  
  -L "127.0.0.1:18081:$HA_VIP:$HA_PORT" \  
  "${VIDEO2_SSH_USER:-root}@${LB1_SSH_HOST:?fill LB1_SSH_HOST}" &  
VIP_TUNNEL_PID=$!  
printf '%s\n' "$VIP_TUNNEL_PID" > "$HA_DIR/.vip-tunnel.pid"  
sleep 2  
curl -fsS http://127.0.0.1:18081/readyz  
open http://127.0.0.1:18081/
```

✅ После команды

Ожидая: оба сервиса active, VIP есть ровно на одном LB, запрос через VIP возвращает HTTP 200. Браузер открывает тот же private VIP через локальный SSH-туннель; это мост лабы, а не public ingress.

⚠ ОГРАНИЧЕНИЕ

VIP `10.77.0.10` живёт внутри лабораторного VXLAN пары LB. Это не публичный cloud IP. В production внешний адрес, маршрутизация и VRRP/BGP зависят от сети площадки; иногда failover делают облачным LB или DNS, но у DNS другое время переключения и кэширование.

ПЕРЕХОД

«Схема работает. В части 1 мы уже ломали пару как отдельную HA-лабораторию. Здесь повторяю отказ не ради второй установки, а чтобы связать его с полной клиентской цепочкой через браузерный туннель. Снова определяю фактического владельца VIP и останавливаю его HAProxy».

Время: 118:00–130:00

ЭКРАН: owner detection → секундная probe → stop HAProxy → VIP/journal/probe.

ГОВОРЮ

«Из-за `nopreempt` активным может быть lb1 или lb2. Поэтому ищу VIP на обоих и сохраняю имя владельца. Параллельно запускаю секундную HTTP-пробу с lb1».

📋 **V2-C4-S12-C01 · Определить текущего владельца VIP и запустить измеряемую пробу**

```
VIP_OWNERS=()
for host in lb1 lb2; do
    if ansible -i "$INV" "$host" --private-key "$SSH_KEY" --become \
        -m ansible.builtin.command -a 'ip -4 addr show dev vxlan100' | grep -q
"$HA_VIP/24"; then
        VIP_OWNERS+=("$host")
    fi
done
test "${#VIP_OWNERS[@]}" -eq 1
ACTIVE_LB="${VIP_OWNERS[0]}"
printf '%s\n' "$ACTIVE_LB" > "$HA_DIR/.failed-active-host"
printf 'VIP owner before failure: %s\n' "$ACTIVE_LB"
ansible -i "$INV" lb1 --private-key "$SSH_KEY" --become -m ansible.builtin.shell \
    -a "rm -f /tmp/vip-probe.log; nohup sh -c 'for i in \$(seq 1 25); do printf
\"%s \" \"\$(date +%T)\"; curl --max-time 2 -sS -o /dev/null -w \"%
{http_code}\\n\" http://$HA_VIP:$HA_PORT/readyz || echo timeout; sleep 1; done'
>/tmp/vip-probe.log 2>&1 </dev/null &"
```


✅ После команды

Ожидаю: владелец определяется фактом, а не предположением `lb1`; на lb1 в фоне началась секундная HTTP-проба.

🗣 ГОВОРЮ

«Останавливаю только HAProxy активного LB. `track_script` замечает отказ, `keepalived` меняет состояние, второй LB забирает VIP. Bounded-цикл ждёт единственного нового владельца и HTTP 200; фиксированную паузу не угадываю. Фактический разрыв читаю из `probe log`».

📄 V2-C4-S12-C02 · Остановить HAProxy активного LB и доказать переезд VIP

```
ansible -i "$INV" "$ACTIVE_LB" --private-key "$SSH_KEY" --become \
  -m ansible.builtin.command -a 'systemctl stop haproxy'
NEW_OWNER=''
for _ in $(seq 1 30); do
  VIP_OWNERS=()
  for host in lb1 lb2; do
    if ansible -i "$INV" "$host" --private-key "$SSH_KEY" --become \
      -m ansible.builtin.command -a 'ip -4 addr show dev vxlan100' \
      | grep -q "$HA_VIP/24"; then
      VIP_OWNERS+=("$host")
    fi
  done
  if [ "${#VIP_OWNERS[@]}" -eq 1 ] \
    && [ "${VIP_OWNERS[0]}" != "$ACTIVE_LB" ] \
    && ansible -i "$INV" lb1 --private-key "$SSH_KEY" --become \
      -m ansible.builtin.uri \
      -a "url=http://$HA_VIP:$HA_PORT/readyz status_code=200" \
      >/dev/null 2>&1; then
    NEW_OWNER="${VIP_OWNERS[0]}"
    break
  fi
  sleep 1
done
test -n "$NEW_OWNER"
printf 'VIP owner after failure: %s\n' "$NEW_OWNER"
ansible -i "$INV" load_balancers --private-key "$SSH_KEY" --become \
  -m ansible.builtin.command -a 'ip -4 addr show dev vxlan100'
ansible -i "$INV" load_balancers --private-key "$SSH_KEY" --become \
  -m ansible.builtin.shell \
  -a "journalctl -u keepalived --since '3 minutes ago' --no-pager | tail -24"
ansible -i "$INV" lb1 --private-key "$SSH_KEY" --become \
  -m ansible.builtin.command -a 'cat /tmp/vip-probe.log'
```

```
ansible -i "$INV" lb1 --private-key "$SSH_KEY" --become \  
-m ansible.builtin.uri -a "url=http://$HA_VIP:$HA_PORT/readyz status_code=200"
```

✅ После команды

Ожидаю: VIP находится на втором LB, журнал фиксирует смену состояния, в probe виден фактический краткий разрыв/timeout и последующие 200; итоговый HTTP снова 200.

🧠 ДОКАЗАТЕЛЬСТВА

Одновременно нужны четыре факта: HAProxy действительно остановлен на прежнем владельце; VIP исчез там и появился на втором; keepralived journal показывает переход; клиент после разрыва снова получает 200.

ПЕРЕХОД

«Демонстрация отказа не закончена, пока стенд не восстановлен».

Время: 130:00–140:00

ЭКРАН: start HAProxy → оба active → VIP один → HTTP 200 → финальная доска.



ГОВОРЮ

«Читаю имя отказавшего LB из локального marker, запускаю HAProxy и повторяю три проверки. Из-за `no-preempt` VIP не обязан вернуться на lb1. Правильный критерий: оба HAProxy и keepalived active, VIP ровно на одном LB, HTTP 200».

📑 V2-C4-S13-C01 · Восстановить отказавший LB без требования вернуть VIP на lb1

```
FAILED_LB="$(cat "$HA_DIR/.failed-active-host")"
[[ "$FAILED_LB" == "lb1" || "$FAILED_LB" == "lb2" ]]
ansible -i "$INV" "$FAILED_LB" --private-key "$SSH_KEY" --become \
    -m ansible.builtin.command -a 'systemctl start haproxy'
ansible -i "$INV" load_balancers --private-key "$SSH_KEY" --become \
    -m ansible.builtin.command -a 'systemctl is-active haproxy keepalived'
ansible -i "$INV" load_balancers --private-key "$SSH_KEY" --become \
    -m ansible.builtin.command -a 'ip -4 addr show dev vxlan100'
VIP_OWNERS=()
for host in lb1 lb2; do
    if ansible -i "$INV" "$host" --private-key "$SSH_KEY" --become \
        -m ansible.builtin.command -a 'ip -4 addr show dev vxlan100' \
        | grep -q "$HA_VIP/24"; then
        VIP_OWNERS+=("$host")
    fi
done
test "${#VIP_OWNERS[@]}" -eq 1
ansible -i "$INV" lb1 --private-key "$SSH_KEY" --become \
    -m ansible.builtin.uri -a "url=http://$HA_VIP:$HA_PORT/readyz status_code=200"
curl -fsS http://127.0.0.1:18081/readyz
VIP_TUNNEL_PID="$(cat "$HA_DIR/.vip-tunnel.pid" 2>/dev/null || true)"
[[ -z "$VIP_TUNNEL_PID" ]] || kill "$VIP_TUNNEL_PID" 2>/dev/null || true
rm -f "$HA_DIR/.vip-tunnel.pid"
rm -f "$HA_DIR/.failed-active-host"
kubectl --context "$CTX" -n traffic-lab delete pod client --ignore-not-found
```

✅ После команды

Ожидая: HAProxy и keepalived active на обоих LB, VIP ровно на одном, HTTP 200 и с LB, и через браузерный туннель. Из-за `no-preempt` VIP не обязан вернуться на lb1; туннель и debug-client удалены.

ГОВОРЮ

«Теперь весь лабораторный маршрут подкреплён фактами. Браузер идёт через локальный SSH-туннель к private VIP. keepalived держит один адрес на живом внешнем LB. HAProxy L4 выбирает healthy private NodePort. Kubernetes Service связывает этот вход с ready адресами EndpointSlice. kube-proxy реализует текущий IPVS data plane. На конце отвечает Pod DEVOPS MAY CRY. В production вместо SSH-туннеля будет public адрес и реальная маршрутизация.

Если запрос пропал, не надо перезапускать всё подряд. Идём по тем же точкам: DNS и route → VIP owner → HAProxy health → NodePort → Service selector → EndpointSlice → Pod readiness. На каждом шаге есть команда и наблюдаемый результат».

ФИНАЛ

«Мы начали с Linux-машин и дошли до отказоустойчивого внешнего входа. Между ними были Terraform, Ansible, Kubespray, Scheduler, cgroups, autoscaling, DNS и Service data plane. Все исходники и лаборатории лежат в одном репозитории; команды можно повторить без скрытых install-скриптов».